ASSESSMENT:	GROUP ASSIGNMENT	**PROGRAMME:**	Diploma in ICT
MODULE CODE:	NWED 513	**MARKS:**	100
MODULE DESCRIPTION:	Web Development 1	**DATE:**	**28 February 2026**
EXAMINER(S):	Melvin Kisten	**DUE DATE:**	**30 March 2026 (NO EXTENSIONS)**
GROUP SIZE:	8 to 10 students		

INSTRUCTIONS:

- Answer ALL questions.
- Indicate your **full names and student number** on your **answer sheet, code, etc.**
- **Copying from a fellow student or any source will be treated as plagiarism.**
- **Submission Requirements**
- **Use Submission template or your submission will not be graded**
- **Report (3 to 5 pages) DOCX and PDF**
- **Video demo**
- Create dedicated GitHub repository, include:
  - Comprehensive codebase or HTML, CSS, JavaScript
  - Execution instructions
  - Environment configuration
  - Maintain clean, commented code
  - Use version control **(git):**
  - **By forking this repo:** https://github.com/login?return_to=%2Fiammelvink%2FNWED51
  - Implement error handling

Group Assignment – Developing Websites:

Make a working system that:

- Fetches and displays data from a public, free REST API
- Uses clean, semantic HTML, modern, responsive CSS (like Flexbox or Grid), and modular, asynchronous JavaScript
- Has clean code with Git version control
- Shows error handling
- Includes testing and a clear report
- Ends with a demo

Deliverables:

1. **Problem Definition (10 marks)**
    - Write a short problem statement
    - Explain why it matters in ICT
    - Show your plan and resources
2. **System Design (20 marks)**
    - User: Interacts with the Browser
    - Frontend Client (Your Website): HTML/CSS/JavaScript running in the browser
    - External REST API: The third-party service (e.g., OpenWeatherMap API)
    - Data Flow: User action -> JS fetch() -> API -> JSON response -> JS DOM manipulation -> Updated UI
    - Document setup and tools used
3. **Implementation (20 marks)**
    - HTML: Create a well-structured, semantic layout
    - CSS: Implement a responsive design that works on mobile, tablet, and desktop. Use Flexbox or Grid
    - JavaScript: Write modular code
    - Use GitHub for version control
    - Write clear, modular code with comments
    - Handle errors properly
4. **Testing & Evaluation (20 marks)**
    - Create a test plan
    - Run unit tests
    - Check performance and scalability

5. **Report (20 marks)**
    - Include background and context
    - Explain your methods
        - Include screenshots of your architecture diagram and final website
    - Show results and discuss them
        - Show screenshots of your final website on different screen sizes
        - Present your testing results (the table from Q4) and Lighthouse scores
        - Discuss any challenges you faced (e.g., CORS issues, understanding asynchronous JavaScript) and how you overcame them
    - List limitations and future improvements
6. **Code & Demo (10 marks)**
    - Show how your system works
    - Answer questions clearly
    - Use simple visuals

Marking Rubric (100 Marks)

Criterion	What to Look For	Score Range	Quick Check Questions
Problem Definition (10)	Does the report clearly describe a simple, relevant problem for a frontend app? Does it explain why presenting data from an API is an important ICT skill?	1-10	"Can I understand the website's purpose and its relevance from the first page?"
System Design (20)	Does the report include a clear diagram showing the user, the browser, and the external API? Are the specific API endpoints and data flows named?	1-20	"Can I follow how data moves from the API to the user's screen based on the diagram?"
Implementation (20)	Do screenshots and the GitHub repo show a working, responsive website? Is there clear, modular JavaScript with fetch and error handling?	1-20	"Do I see a functional website with structured code that handles errors gracefully?"
Testing & Evaluation (20)	Does the report include a clear test plan with results? Are there responsiveness checks?	1-20	"Do they show they tested their website's features and performance?"

Report (20)	Is the report well-structured, telling the story from problem to solution, with clear methods, results (screenshots, test tables), limitations, and future ideas?	1-20	"Could a general reader follow their entire project journey?"
Code & Demo (10)	Is there a short demo video? Is the GitHub repo well-organised with a good README and clean code?	1-10	"Is there clear evidence of a working system and a well-managed codebase?"

Criteria	**Excellent (8-10 / 16-20)**	**Good (5-7 / 10-15)**	**Poor (1-4 / 1-9)**
Q1: Problem Definition (10)	Clear, simple, relevant problem	Minor gaps, basic context	Vague or unclear
Q2: System Design (20)	Full diagram, clear endpoints	Basic diagram, missing some	No clear design
Q3: Implementation (20)	APIs, Git, clean code, errors handled	Works but untidy, some missing parts	Broken or incomplete
Q4: Testing (20)	Test plan, unit tests, metrics	Basic tests, few metrics	No tests
Q5: Report (20)	Well-structured, clear results, defence	Limited details, average	Unclear or missing
Q6: Code & Demo (10)	Clean code, smooth demo	Some issues, average demo	Messy or no demo

Criteria	**Excellent**	**Good**	**Poor**
Q1: Problem Definition (10)	8-10	5-7	1-4
Q2: System Design (20)	16-20	10-15	1-9
Q3: Implementation (20)	16-20	10-15	1-9
Q4: Testing (20)	16-20	10-15	1-9
Q5: Report (20)	16-20	10-15	1-9
Q6: Code & Demo (10)	8-10	5-7	1-4

---------- All the best ----------